Scaffold

A Real-Time Collaborative Drawing Platform for Artists

Product Requirements Document — Version 1

Prepared by DevHub Labs

Product name finalized

1. Overview

1.1 Product Summary

Scaffold is a real-time collaborative drawing web application built for manga and comic artists — and for anyone who has a character in their head but doesn't yet know how to get it on paper. Multiple users can draw on the same canvas simultaneously, from different devices (mouse, touch, or stylus/pen), see each other's work live, and talk to one another by voice while they work.

The product's defining principle: the tool handles technical, time-consuming groundwork — construction shapes, proportion guides, panel layout — while leaving everything that makes the art personal (linework, style, expression, character design) entirely in the artist's hands. Scaffold does not generate finished art on a user's behalf — it builds the scaffold; the artist builds the art.

1.2 Problem Statement

- Manga and comic creation involves significant "invisible" preparatory work — rough construction shapes, proportion scaffolding, and panel layout — before expressive linework begins. This slows artists down without adding creative value.
- Existing AI drawing tools typically solve this by generating finished art for the user. This undermines originality and causes visual homogenization — art produced with these tools tends to look alike, and can morph an artist's intended characters and faces.
- Manga is frequently produced by small teams (e.g., one artist on layout, another on inking) who currently collaborate by manually passing files back and forth, with no live shared workspace and no built-in communication.

1.3 Goals

- Provide a genuinely useful, real-time collaborative canvas for manga/comic artists.
- Reduce time spent on construction/proportion work without generating art for the user or eroding their originality.
- Support natural drawing input across mouse, touch, and stylus, including pressure sensitivity.
- Enable remote teams to draw and communicate on the same page at the same time.
- Ship a working, well-tested v1 at a sustainable pace, documented publicly as a build-in-public project.

1.4 Non-Goals (for v1)

- Generating finished, final artwork or characters on the user's behalf.
- Per-user isolated undo history (v1 undo is shared/global).
- A native mobile application (v1 is a web app, touch-friendly but not app-store-polished).
- Heavy scaling infrastructure (e.g., Redis) ahead of actual need.
- Blockchain/decentralization features (parked on the roadmap).

2. Target Users & Use Cases

2.1 Primary Users

- Manga/comic artists working solo who want faster, cleaner construction work without losing their personal style.
- Small manga/comic teams (e.g., a layout artist and an inker) who need to co-create a page remotely, in real time.
- Aspiring or learning artists who have a character concept but lack confidence or skill to draft proportions and structure independently.

2.2 Key Use Cases

Use Case	Description
Solo drafting	An artist opens a room alone, sketches a manga page, uses Snap-to-Shape to clean up construction lines, and exports the finished page.
Remote co-creation	Two or more artists join the same room from different locations, draw on the same page live, and talk via built-in voice call while working.
Guided drafting	A user with limited drawing experience describes a character (age, body type, gender) and receives an editable proportion scaffold to build on.
Cross-device drafting	A user without a stylus roughly sketches small text or a diagram with a finger, then selects and scales it to the desired size.

3. Features & Requirements (v1)

3.1 Drawing & Canvas

- Support drawing input via mouse, touch, and stylus/pen using a unified input model (Pointer Events API).
- Pressure-sensitive line weight where hardware supports it; fixed-width fallback for mouse or plain touch.
- Disambiguate drawing gestures from pan/zoom gestures on touch devices.
- Basic palm rejection when a stylus is actively in use.
- Multiple brush types: pen, pencil, eraser, ink brush.
- Layer support, minimum of a sketch layer and an ink layer.
- Manga-style page templates with panel borders.
- Undo/redo (shared/global stack for v1).
- Export a page as PNG or PDF.

3.2 Snap-to-Shape

- Detect rough circle, ellipse, straight line, and rectangle strokes in real time.
- Clean the detected shape into a precise version of the user's own stroke, computed client-side using geometry/curve fitting — no generative model involved.
- Feature must be toggleable on/off per user.

3.3 Select & Scale

- Allow a user to select a drawn element (e.g., small or imprecise text or a diagram) and resize it to a desired scale.
- Primarily intended to support users on touch devices without a stylus.

3.4 Collaboration

- Create a room and share a link for others to join.
- Real-time broadcast of drawing strokes and cursor positions to all users in a room.
- Distinct, labeled cursors per active collaborator.
- Live voice call available to all users within a room.
- Automatic saving of room/canvas state so users can leave and resume later.

3.5 Co-Artist Mode

Co-Artist Mode is split into three sub-features with materially different levels of technical risk. Each is scoped and staged independently rather than treated as a single feature.

3.5.1 Text → Proportion Scaffold

- User provides basic descriptive inputs (e.g., age range, body type, gender).
- System generates a proportion scaffold (head oval, body height guides, shoulder/hip ratios) as real, editable canvas objects — not a locked overlay image.
- Implementation: parametric geometry generation using established figure-drawing ratios; no machine learning model required.
- Risk level: Low. This is scoped as buildable, known engineering.

3.5.2 Image → Step-by-Step Drawing Sequence

- User uploads a reference image; system extracts a pose/skeleton and walks the user through a construction sequence (basic shapes through structure).
- Implementation approach: pose extraction via an existing model (e.g., MediaPipe Pose), mapped into a fixed, pre-designed teaching-order template (head → spine → shoulders/hips → limbs → refine), rather than a dynamically-decided teaching order.
- Risk level: High. Automatically decomposing an arbitrary image into an optimal, dynamic teaching sequence is an open, unsolved problem; scoping this to a fixed template converts it into a tractable engineering task, but this sub-feature should be treated as exploratory until validated.

3.5.3 Live Natural-Language Canvas Editing

- User issues commands such as "make her taller" or "sharper jaw," and the relevant canvas objects update live for all users in the room.
- Implementation approach: an LLM parses commands into a fixed, constrained set of adjustable parameters (e.g., height, shoulder width, hip width, jaw angle) rather than fully open-ended freeform edits.
- Risk level: High. Requires a bespoke mapping layer between natural language and specific canvas transforms; scoping to a fixed parameter set is what makes this achievable for v1.

Team consistency principle: Character consistency across a collaborating team is achieved by having all collaborators build on the same AI-generated scaffold, combined with Snap-to-Shape smoothing — not by the AI enforcing sameness after the fact.

3.6 Security & Access

- Token-based room access, restricting room entry to invited users.
- Secure handling of auth/session tokens.

4. Non-Functional Requirements

Category	Requirement
Performance	Drawing and cursor sync must feel responsive across users in the same room; local strokes render immediately without waiting on server confirmation.
Cross-device	Coordinate handling must normalize across differing screen sizes and resolutions; input handling must behave consistently across mouse, touch, and stylus.
Reliability	Canvas/room state must be periodically persisted so refreshes or disconnects do not cause data loss.
Scalability	v1 infrastructure is intentionally simple (in-memory state plus periodic database snapshot); heavier infrastructure (e.g., Redis) is deferred until real scale need is demonstrated.
Accessibility of collaboration	Voice call and cursors should make remote collaboration feel close to in-person collaboration.

5. Technical Architecture

5.1 Frontend

- React (UI framework)
- Fabric.js on HTML5 Canvas (canvas engine, object model, layers, transforms)
- Pointer Events API (unified mouse/touch/stylus input, pressure/tilt support)
- Custom JavaScript geometry fitting for Snap-to-Shape (no external library or model)
- Socket.io client for real-time connectivity

5.2 Backend

Backend responsibilities are split by language according to team strengths.

Python (FastAPI) — core real-time and AI-adjacent logic

- Room management and session handling
- Real-time sync via python-socketio (broadcasting draw events and cursor positions)
- Co-Artist backend: text-to-scaffold generation, MediaPipe pose extraction, NLP command parsing

JavaScript (Node.js/Express) — auxiliary services and integration

- Auth/session REST endpoints
- Export service (canvas state to PNG/PDF)
- Glue layer between the React frontend and Python real-time/AI services

5.3 Data & Infrastructure

- Database: MongoDB (accessed via motor for async Python), storing canvas/stroke state, room data, and saved projects
- Voice call: hosted WebRTC service (e.g., Daily.co or Agora) or PeerJS
- Hosting: Render or Railway

6. Pages & Screens

Page/Screen	Purpose
Landing / Home	Introduces the product; entry points to create or join a room.
Auth	Sign in / sign up, required for saved projects and token-based room access.
Dashboard	Displays a user's saved projects/rooms; entry point for starting a new project.
Room Creation / Join	Generate a shareable room link, or enter one to join an existing room.
Canvas Workspace	The core, most complex screen, comprising: — Canvas — Toolbar (brushes, colors, Snap-to-Shape toggle, scale tool) — Layers panel — Co-Artist side panel (text/image input, live command box) — Collaborator list, voice call controls, live cursors
Export / Save	Screen or modal for PNG/PDF export options.

7. Design System

Finalized and ready for implementation. Davis and sekibo build all UI/UX work directly against these values.

7.1 Colors

- Background: #16171C (main app background, dark neutral)
- Panel: #1F2128 (toolbar/sidebar/panel background, slightly lighter than base)
- Border: #2E313A (dividers, outlines)
- Text primary: #F2F2F0 (main text on dark background)
- Text muted: #8B8D96 (secondary text, labels, captions)
- Accent: #3D7EDB (buttons, active states, links)
- Accent hover: #5A93E6

7.2 Collaborator Cursor Colors

Assigned one per active user in a room; cycle back to the first if more than six users join simultaneously.

- #E8544E, #3DB88C, #E6B33D, #A66DE0, #3D7EDB, #E67E3D

7.3 Typography

- Font: Inter (Google Fonts) — fallback: system-ui, sans-serif
- Headings: medium weight (500–600), not bold — keeps the interface quiet next to hand-drawn artwork
- Body text: 16px, line-height 1.5

7.4 Conventions

- Border radius: 6px on buttons and cards — soft but not overly rounded
- Spacing scale: multiples of 4px (8, 12, 16, 24, 32) for all margins/padding, to keep separately-built screens visually consistent
- Buttons: accent background, white text, 6px radius, 10px/20px padding, hover to accent-hover
- Cards/panels: panel-color background, 1px border in border-color, 6px radius

7.5 Base CSS

Reference starting point for Davis and sekibo:

- Root variables for all colors above, applied as CSS custom properties
- Body: background var(--bg), color var(--text-primary), font-family var(--font), 16px, line-height 1.5

7.6 Page-by-Page Guidance

Landing / Home — Davis

- Full-height dark background, centered content
- App name/logo top-left or centered top
- Short one-line pitch text in muted color, smaller size
- Two large CTAs: "Create Room" (accent-filled) and "Join Room" (outlined, accent border, transparent background)
- Keep sparse — this screen should communicate the idea within seconds

Auth — sekibo

- Centered card (panel background, 6px radius, ~32px padding) on the dark page background
- Tab/toggle at top: "Sign In" / "Sign Up"
- Stacked fields: email, password, confirm-password (sign-up only); label above each input; input background #25272F, border-color outline, 6px radius
- Full-width accent button at the bottom

- Small muted-text link to switch between Sign In / Sign Up

Dashboard — Davis

- Top bar: app name/logo left, user avatar/menu right
- Grid or list of project cards: thumbnail placeholder (solid panel-color block), project name, last-edited date in muted text
- Clearly visible "+ New Project" card or button, matching Landing's accent styling

Room Creation / Join — sekibo

- Two clear sections or a tab switch: "Create a Room" and "Join a Room"
- Create: button generates a room link (placeholder text acceptable for now) plus a copy-link button
- Join: single input field for a room code/link plus an accent "Join" button
- Keep the two sections visually separated with spacing or a divider

Export / Save — Davis

- Simple modal or dedicated screen
- Preview thumbnail placeholder of the canvas (panel-colored block is sufficient for now)
- Two equal-size, side-by-side accent-outlined buttons: "Export as PNG" and "Export as PDF"
- Small muted confirmation text area below, shown after export (can be static/fake until backend is wired)

Canvas Workspace UI Markup — sekibo (structure/CSS only, no live data)

- Toolbar: horizontal strip (top or left), panel background — brush, eraser, color swatch, brush size, Snap-to-Shape toggle, Scale tool, evenly spaced, icon placeholders acceptable
- Layers panel: narrow fixed sidebar (right is conventional), panel background, rows of visibility-toggle icon plus layer name, "+ Add Layer" button at the bottom
- Collaborator list: small strip in the top-right corner, stacked circular avatars with a colored ring matching each user's cursor color, plus a mic icon/button for voice call
- This is pure structure and styling — no live data or working toggles. Obi wires in real behavior afterward.

8. Team Roles & Ownership

This section reflects the final, confirmed allocation as of team kickoff. All v1 features have a named owner; no items remain open.

Role	Owner	Scope
Canvas Engine + Real-Time Client	Obi	Fabric.js canvas, Pointer Events input (mouse/touch/stylus + pressure), gesture disambiguation, palm rejection, brushes, layers, undo/redo, Snap-to-Shape integration, Select & Scale, Socket.io client integration (cursor rendering, live sync).
Python / FastAPI Core	Ronald	Room management, real-time sync (python-socketio), stroke/cursor broadcast, auto-save/persistence, voice call (hosted WebRTC), Co-Artist backend (lowest priority).
JS Backend + Content Lead	Testimony	Auth/session endpoints, room-access security (JWT-based), export service (PNG/PDF), frontend–backend integration layer, content pipeline (collecting clips, editing, posting).
UI/UX — Screens	Davis	All static screens in HTML/CSS/basic JS: Landing, Auth, Dashboard, Room Creation/Join, Export/Save — built from the design system (Section 7).
Fabric.js Panel Templates + Canvas Workspace UI	sekibo	Manga panel/page border templates (Fabric.js); Canvas Workspace screen markup (toolbar, layers panel, collaborator list structure) — sequenced after Davis has shipped 1–2 screens.

Note: Emmanuel was originally allocated Design Direction and React UI/Socket.io client work, but is no longer participating in the project due to a longer-term commitment elsewhere. His scope has been fully redistributed: the design system was produced directly for the team (Section 7) rather than left as an open dependency, UI/screen work moved to Davis and sekibo, and Socket.io client work is now a permanent part of Obi's role.

Ronald's role carries the highest technical load in the project (real-time sync plus all AI-adjacent Co-Artist work). His content involvement is intentionally limited to documenting his own progress; the overall content pipeline is owned by Testimony, to avoid overloading the most heavily-tasked contributor.

Obi's role is the largest in scope (Canvas Engine plus real-time client integration). This was accepted knowingly given Canvas Engine is the single biggest piece of the product; the team should stay alert to this balance as the project progresses.

9. Backend Architecture & Shared Contract

Ronald (Python/FastAPI) and Testimony (Node.js/Express) work as two separate services, not a shared codebase. The seam between them is the highest-risk integration point in the project and must be agreed before either side builds deep logic against assumptions.

9.1 Folder Structure

- backend-python/ (Ronald) — main.py, rooms.py, sockets.py, coartist.py, requirements.txt
- backend-node/ (Testimony) — server.js, routes/auth.js, routes/export.js, middleware/roomAuth.js, package.json
- frontend/ (Obi, Davis, sekibo) — React app
- Each service runs independently on its own local port (e.g., Python on 8000, Node on 4000) and communicates only over HTTP/WebSocket — no shared runtime.

9.2 Shared Data Contract

Both services build against these exact shapes. Field names are not to be changed unilaterally once frontend work depends on them — any change must be raised with the team first.

- stroke: { type: "stroke", roomId, userId, points: [[x,y], ...], color, width } — sent client to Ronald's server, broadcast to room
- cursor: { type: "cursor", roomId, userId, x, y } — sent frequently, broadcast to room
- join-room: { type: "join-room", roomId, userId, authToken }
- Auth token: a JWT issued by Testimony's Node service on login; sent by the frontend when joining a room; verified by Ronald's Python service before allowing a socket connection.

9.3 Ronald's Build Order

- Minimal FastAPI WebSocket endpoint that echoes messages back, to prove connectivity
- Room creation/join logic using the join-room shape above
- Broadcast cursor events to all other users in a room
- Broadcast stroke events the same way, once Obi's canvas produces real stroke data
- Auto-save — periodic snapshot of room canvas state to MongoDB
- Voice call (hosted WebRTC — Daily.co or Agora)
- Co-Artist backend — last, highest risk, lowest priority

9.4 Testimony's Build Order

- Auth/session REST endpoints (signup, login) — issues the JWT used in the shared contract; independent, starts immediately

- Room-access security — middleware verifying a valid token before a user reaches Ronald's socket layer; requires a short sync with Ronald on token format
- Export service (canvas state to PNG/PDF) — once Obi's canvas exists
- Integration/glue layer — anything the frontend needs sitting between React and Ronald's Python service

9.5 Required Early Sync

Before either service goes too deep: a 15–20 minute call between Ronald and Testimony to confirm the JSON shapes above, the JWT format and verification point, and each service's local port — cheaper to align now than to debug a silent mismatch later.

10. Build Sequence & Dependencies

All five team members start on Day 1 in parallel — no one is blocked waiting on another person's output.

Owner	Day 1 task	Depends on
Obi	Pointer Events integration (touch/stylus + pressure)	Nothing — canvas already draws
Ronald	Minimal FastAPI WebSocket echo endpoint	Nothing
Testimony	Auth/session REST endpoints	Nothing
Davis	Landing page (HTML/CSS)	Nothing — design system is finalized
sekibo	Panel/page templates (Fabric.js)	Nothing

10.1 What Comes Next, and Why

- sekibo — Canvas Workspace UI markup: starts once Davis has shipped 1–2 screens, so there's a proven pattern to follow
- Testimony — Export service: starts once Obi's canvas is functional with layers/brushes
- Ronald — Stroke broadcast: starts once Obi's canvas is sending real stroke data
- Obi — Socket.io client wiring: starts once Ronald's room/broadcast logic is running
- Ronald — Voice call and Co-Artist backend: scheduled last, after core real-time sync is solid
- Testimony — Room-access security: can be scaffolded in parallel, finalized after the JWT sync with Ronald

Recommendation: keep a recurring check-in (weekly or biweekly) focused specifically on the handoff points — Obi's canvas to Ronald's sync layer, Ronald's backend to Testimony's integration layer, Davis's shipped screens to sekibo's Canvas Workspace markup — since these connection points, not individual task lists, are the most likely place for the project to stall.

11. Risks & Open Questions

Risk	Mitigation
Co-Artist's image-to-sequence and NLP-editing features are technically ambitious and could stall the whole v1 if treated as blocking.	Scoped and scheduled last in the build sequence; core product (canvas, collaboration, Snap-to-Shape) is fully shippable independent of these.
Obi's role is the largest in scope (Canvas Engine plus real-time client integration), following Emmanuel's departure.	Explicitly flagged to the team; revisit redistributing a piece of this (e.g., Socket.io client work) to Davis or sekibo once they're comfortable with their current tasks.
Palm rejection and touch/stylus edge cases are genuinely difficult on the web.	Basic palm rejection scoped as best-effort, not guaranteed-perfect, for v1; cross-device testing to be folded into the team's regular check-ins.
Team has a history of stalling on ambitious features (per prior project ECHO).	Explicit non-goals section, staged rollout, named ownership for every v1 feature, and recurring handoff check-ins built into this PRD.

Risk	Mitigation
Python (Ronald) and Node (Testimony) backends are separate services with no shared runtime.	Shared data contract defined in Section 9.2; required early sync between Ronald and Testimony before deep integration work begins.

12. Working Approach

- This project is not being built under hackathon time pressure; pace is deliberately sustainable ("small small").
- The build process is documented publicly as build-in-public content — including setbacks and debates, not only finished results.
- Documenting one's own progress is considered part of each role's job, not a separate task added afterward.